

Day 4

Karnish

(07/08/2025)

1. Create two threads to read two separate text files
 - 1.1 Same two files, try to read using the Task Async await.

Coding:

```
using System;
using System.IO;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static void Main(string[] args)
    {
        string filePath1 = "file3.txt";
        string filePath2 = "file4.txt";

        Console.WriteLine("Sample Thread");

        Thread t1 = new Thread(() => ReadFileWithThread(filePath1));
        Thread t2 = new Thread(() => ReadFileWithThread(filePath2));

        t1.Start();
        t2.Start();

        t1.Join();
        t2.Join();

        Console.WriteLine("\nAsync/Await Example");

        Task task1 = ReadFileAsync(filePath1);
        Task task2 = ReadFileAsync(filePath2);

        Task.WaitAll(task1, task2);

        Console.WriteLine("\nProcess will be done!");
    }

    static void ReadFileWithThread(string path)
    {
        try
        {
            if (!File.Exists(path))
```

```

    {
        File.Create(path).Dispose();
        Console.WriteLine("File created successfully");
        File.WriteAllText(path, "Welcome to programming");
    }
    string content = File.ReadAllText(path);
    Console.WriteLine($"[Thread {Thread.CurrentThread.ManagedThreadId}] Read from
{path}:");
    Console.WriteLine(content);
}
catch (Exception ex)
{
    Console.WriteLine($"Error reading {path}: {ex.Message}");
}
}

static async Task ReadFileAsync(string path)
{
    try
    {
        if (!File.Exists(path))
        {
            File.Create(path).Dispose();
            Console.WriteLine("File will be created successfully");
            File.WriteAllText(path, "New Text Content");
        }
        string content = await File.ReadAllTextAsync(path);
        Console.WriteLine($"Async Read from {path}:");
        Console.WriteLine(content);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error reading {path}: {ex.Message}");
    }
}
}

```

OUTPUT:

```
Sample Thread
File created successfully
File created successfully
[Thread 8] Read from file3.txt:
Welcome to programming
[Thread 9] Read from file4.txt:
Welcome to programming

Async/Await Example
Async Read from file4.txt:
Welcome to programming
Async Read from file3.txt:
Welcome to programming

Process will be done!

D:\newproject\DailyTask\DailyTask\bin\Debug\net8.0\DailyTask.exe (process 19788) exited with code 0 (0x0).
Press any key to close this window . . .
```

2. (Optional) Create delegate use case between teacher class and student class:

- teacher method should have `test_completed()` method passed as delegate to student
- student class should have a method `write_test()` which will inturn call the parent delegate.

CODING:

[Program.cs](#)

```
using System;
```

```
namespace Day4_Task_2 {
    class Program
    {
        static void Main(string[] args)
        {
            Teacher teacher = new Teacher();

            Student student = new Student(teacher.TestCompleted);

            student.WriteTest();
        }
    }
}
```

[Teacher.cs](#)

```
using System;
using System.Collections.Generic;
```

```

using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day4_Task_2
{
    internal class Teacher
    {
        public void TestCompleted()
        {
            Console.WriteLine("Teacher: Test has been completed by the student.");
        }
    }
}

```

[Student.cs](#)

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day4_Task_2
{
    internal class Student
    {
        public delegate void TestDelegate();

        private TestDelegate _testDelegate;

        public Student(TestDelegate testDelegate)
        {
            _testDelegate = testDelegate;
        }

        public void WriteTest()
        {
            Console.WriteLine("Student: Writing the test...");
            _testDelegate?.Invoke();
        }
    }
}

```

OUTPUT:

```
Student: Writing the test...
Teacher: Test has been completed by the student.

D:\newproject\Day4-Task_2\Day4-Task_2\bin\Debug\n
To automatically close the console when debugging
le when debugging stops.
Press any key to close this window . . .
```